

Playwright Automation

Cross-Browser Support

Playwright supports all modern browser engines:

- **Chromium** (Chrome, Edge)
- **WebKit** (Safari engine)
- **Firefox**

Same test works across all browsers without changes.

Cross-Platform Testing

Run tests on:

- Windows
- Linux
- macOS

You can run tests:

- Locally
- On CI/CD pipelines
- In **headless** or **headed** mode

Cross-Language Support

Playwright API is available in:

- TypeScript (recommended)
- JavaScript
- Python
- Java
- .NET (C#)

1.2 Mobile Web Testing

Playwright provides **native mobile emulation**:

- Chrome on Android

- Safari on iPhone (WebKit)

Key point:

- Same rendering engine as real mobile browsers
 - Tests behave close to **real device behavior**
-

1.3 Resilient Tests (No Flakiness)

Auto-Waiting (Very Important)

Playwright automatically waits for:

- Element to be visible
- Element to be enabled
- Element to be stable (no animation)

No need for `sleep()` or hard waits.

Web-First Assertions

Assertions are **retry-based**:

- Playwright keeps checking until condition is met or timeout occurs

Example:

```
await expect(locator).toBeVisible();
```

Tracing & Debugging

Playwright can capture:

- Execution trace
- Screenshots
- Videos

Used to:

- Analyze failures
- Reduce flaky tests

1.4 No Trade-Offs / No Limits

Modern Browser Architecture

- Playwright runs **out-of-process**
- Matches real browser internals
- Avoids Selenium-style limitations

Multiple Everything

Playwright can handle:

- Multiple tabs
- Multiple windows
- Multiple origins (different domains)
- Multiple users (via contexts)

All inside **one test**.

Trusted User Events

- Real hover
- Real clicks
- Real keyboard input

Browser treats actions as **real user actions**, not fake JS events.

Frames & Shadow DOM

- Seamless iframe handling
- Can pierce **Shadow DOM** directly

Shadow DOM is a hidden DOM tree used by modern web components to encapsulate UI elements. Selenium requires manual JavaScript handling to access Shadow DOM, while Playwright supports it natively with auto-waiting and clean selectors, making Playwright better and more reliable for Shadow DOM handling.

No extra hacks required.

1.5 Full Isolation & Fast Execution

Browser Contexts

- Each test gets a **new browser context**
- Equivalent to a fresh browser **profile**
- Zero test data leakage

Playwright uses browser contexts to provide fast, lightweight, and fully isolated browser sessions. Each test runs in its own context, equivalent to a fresh browser profile, which avoids state leakage, flaky, improves stability, and enables parallel execution without opening multiple browser instances.

Browser context in Playwright is a lightweight browser profile that isolates cookies, storage,

Creating a context takes **milliseconds**.

Login Once, Reuse State

- Save authentication state
- Reuse across tests
- Avoid repeated login steps

Still maintains full isolation.

1.6 Powerful Tooling (Big Advantage)

Codegen

- Records user actions
- Generates test code
- Available for all supported languages

Playwright Inspector

- Inspect elements
- Auto-generate selectors
- Step-by-step execution
- View logs & click points

Trace Viewer

Trace contains:

- Video recording
- DOM snapshots
- Action explorer
- Test source code

Best tool to debug failed tests.

1. What is Playwright?

Playwright enables **reliable end-to-end testing** for **modern web applications** using **one API** that works across **all browsers, platforms, and languages**.

In short:

- Any Browser
 - Any Platform
 - One API
-

1.1 Core Capabilities of Playwright

Playwright is an open-source **end-to-end test automation framework** developed by **Microsoft**.

Why Playwright?

- Automates **modern web applications**
- Works with **Chromium, Firefox, WebKit (Safari engine)**
- Very **fast, reliable**, and **auto-waits** for elements
- Supports **UI, API, and performance testing**

Languages Supported

- JavaScript (most common)
 - TypeScript (recommended)
 - Python
 - Java
 - C#
-

2. Playwright vs Selenium

Feature	Playwright	Selenium
Auto wait	Yes (default)	No (manual waits)
Browser support	Chromium, Firefox, WebKit	All major browsers
Speed	Very fast	Slower
Setup	Simple	More configuration
API testing	Built-in	Needs libraries

Feature	Playwright	Selenium	Cypress
iFrame	Very well supported	Supported (switchTo.frame)	Limited
Cross-Origin / Third-Party Login	Very well supported	Supported	NA
Multiple Window / Tab Support	Very well supported	Supported	NA
Multi Language Support	JavaScript, TypeScript, Java, Python, .NET	Java, Python, C#, JavaScript, Ruby	JavaScript, TypeScript
Browser Support	Chrome, Chromium, Firefox, WebKit (Safari)	Chrome, Firefox, Edge, Safari	Chrome, Firefox (dev/nightly), Chromium-based, Edge, Electron
WebKit (Safari)	Very well supported	Supported (SafariDriver – limited)	NA
Speed	Much faster (browser context)	Slower than Playwright	Slow (runs inside browser, CDP APIs)
Page Object Pattern	Very well supported	Industry standard	Not recommended
Parallel Browser Testing	Supported via workers (local + CI) – No license	Supported via Grid / Cloud tools	Needs license
Parallel Support for Tests	Parallel tests in same file	Supported via TestNG / JUnit	Needs splitting tests into files
Async / Await Support	Clean async/await	Not applicable (sync-style API)	Complex chaining
Edge Case Scenarios	In-built features	Custom utilities needed	Needs plugins
Test Grouping & Test Suites	Supported	Supported	NA

Feature	Playwright	Selenium	Cypress
XPath Support	In-built	Native support	Needs plugins
Dependency	Most features in-built	External libraries required	Heavy plugin dependency
Debugging	Inspector, Trace Viewer, Headed & Debug mode	IDE debugging, logs, screenshots	DevTools / own debugger
Automatic Waits	In-built auto-wait	Manual waits required	Limited
Upload / Download Files	In-built features	Supported	Needs plugins
User Authentication	Reuse authentication state	Cookies / profiles	NA
Flake Detection	Retry + trace + video + screenshots	Manual handling	Dashboard (paid)
Pricing	Free	Free	Paid dashboard
Community	Growing	Very mature	Mature

Note:

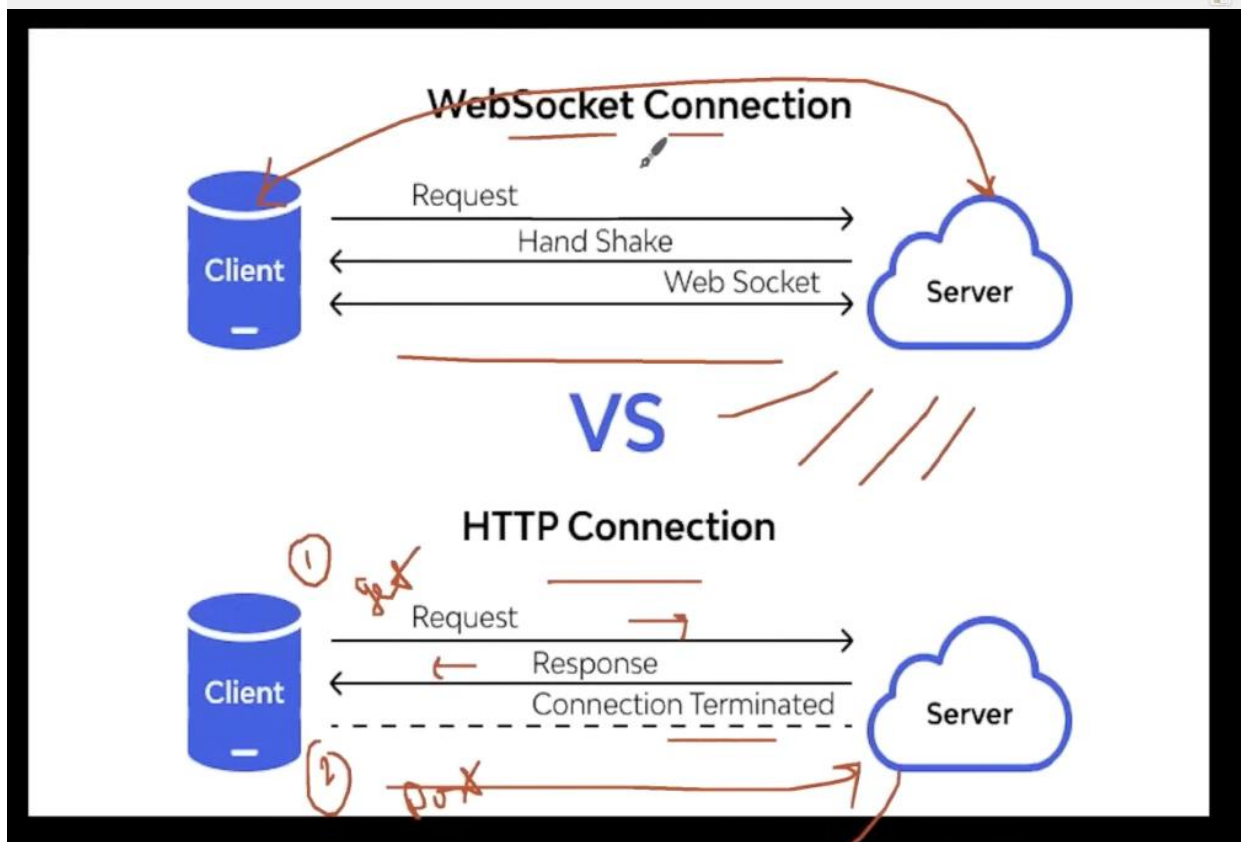
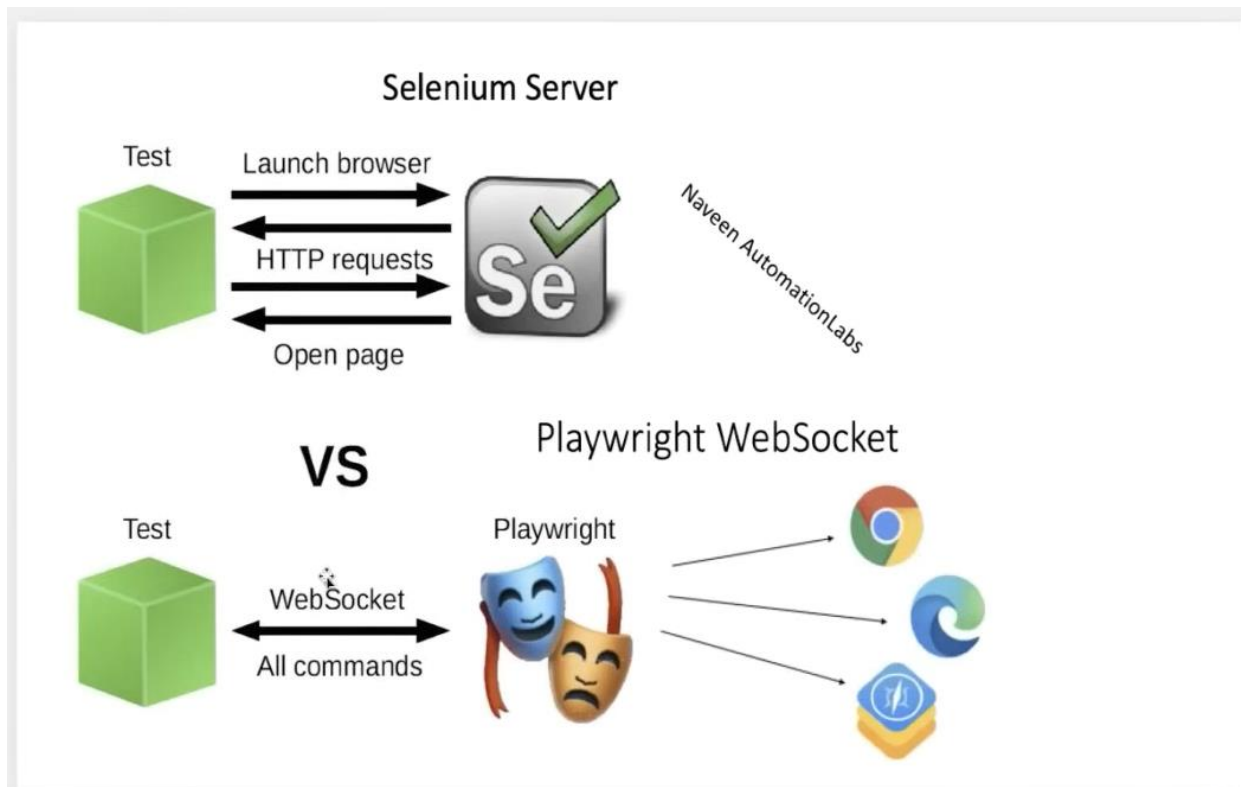
Playwright does not require TestNG or JUnit because it includes its own test runner, assertions, parallel execution, and reporting. TestNG or JUnit can be integrated optionally, but they are not mandatory.

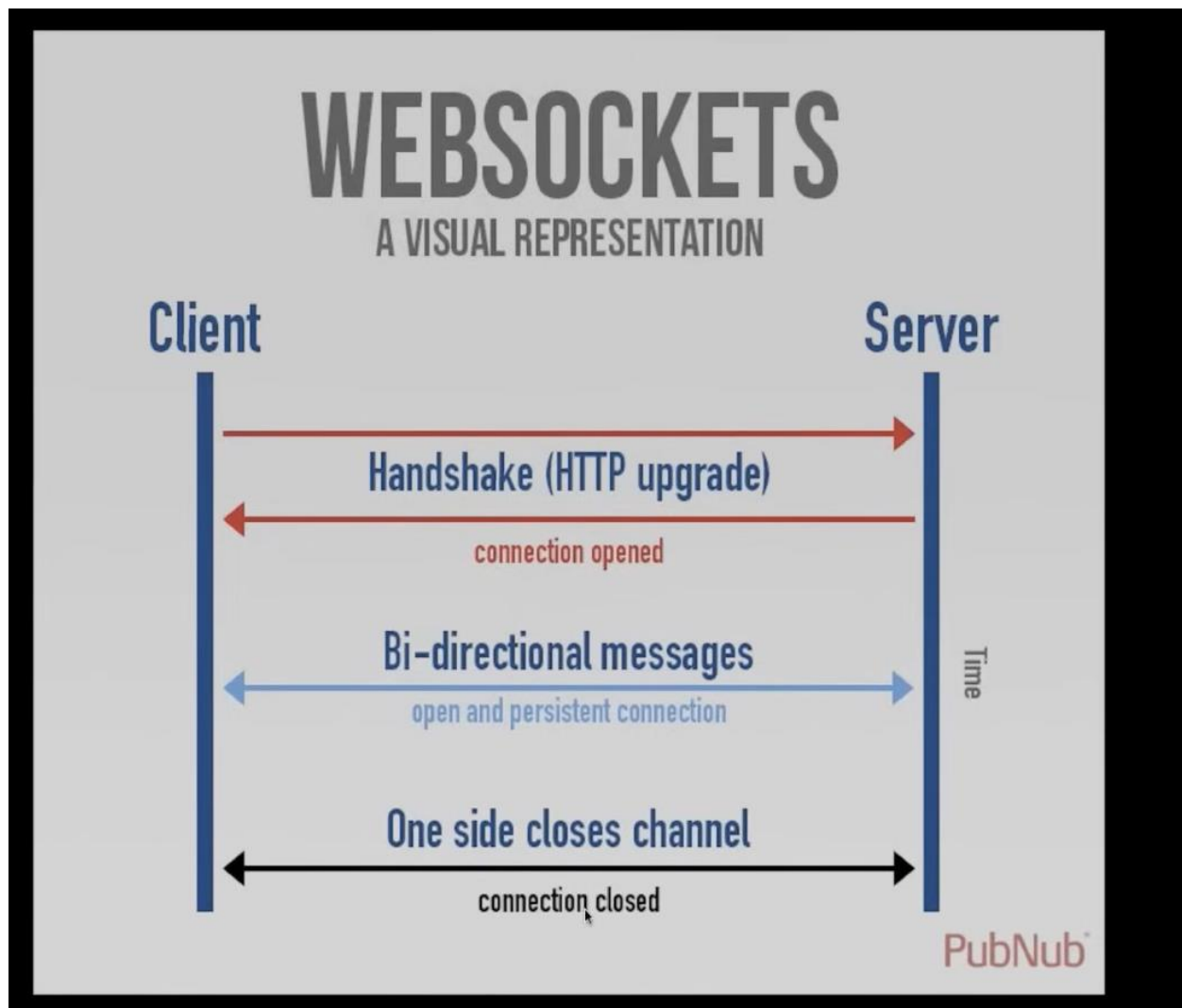
HTTP VS WS(Web Socket):

Selenium uses a stateless HTTP/HTTPS request–response architecture defined by the WebDriver specification, which introduces latency and synchronization challenges. Playwright uses a persistent WebSocket/WSS connection directly with browser engines, enabling real-time event-driven communication, improved synchronization, faster execution, and significantly reduced test flakiness.

HTTP connection terminates once action is completed but ws connection stays.

HTTP connections are short-lived and terminate after each request–response cycle, whereas WebSocket connections are persistent and remain open, enabling real-time, bidirectional communication throughout the test execution.





Binaries:

Browser binaries are the actual executable files required to run browsers or control them during automation.

1. Selenium needs driver binaries (ChromeDriver, GeckoDriver, EdgeDriver)
2. Playwright needs browser binaries (Chromium, Firefox, WebKit)

These binaries act as a bridge between test code and the browser.

Selenium

- Requires **separate driver binaries**

- Tester must:
 - Download drivers manually **OR**
 - Manage them using tools like WebDriverManager
- Version mismatch between browser and driver can break tests

Playwright

- **No separate driver binaries**
- Playwright directly controls browser engines
- Browser binaries are:
 - Chromium
 - Firefox
 - WebKit (Safari engine)

👉 Playwright automatically manages versions internally.

How Playwright handles browser binaries (Java)

When a Playwright Java test runs:

1. Java checks whether the required browser binary is available locally
2. If **not available**, Playwright **automatically downloads** it
3. The downloaded browser is stored inside the Playwright cache
4. Tests run using that browser version

This ensures:

- Version compatibility
- No manual driver management
- Stable execution

Automatic download via Maven (most common)

When you run:

```
mvn test
```

- Playwright checks for required browser binaries
- If missing, Maven + Playwright automatically download them
- No user intervention needed

✅ This is the default and recommended approach

Manual download using command line

You can explicitly download browsers using:

```
npx playwright install
```

or (Java-specific):

```
mvn exec:java -e -Dexec.mainClass=com.microsoft.playwright.CLI -Dexec.args="install"
```

This is useful when:

- Setting up CI/CD
- Pre-downloading browsers
- Running in restricted environments

Note: Playwright manages browser binaries automatically, whereas Selenium requires separate driver binaries, making Playwright easier to maintain and more reliable.

Managing browser binaries

Playwright downloads Chromium, WebKit and Firefox browsers into the OS-specific cache folders:

- `%USERPROFILE%\AppData\Local\ms-playwright` on Windows
- `~/Library/Caches/ms-playwright` on MacOS
- `~/.cache/ms-playwright` on Linux

These browsers will take a few hundred megabytes of disk space when installed:

Chromium Browser:

```
public class BrowserLaunch {  
  
    public static void main(String[] args) {  
  
        Playwright playwright = Playwright.create();  
        Browser browser = playwright.chromium().launch(new  
BrowserType.LaunchOptions().setHeadless(false));  
        BrowserContext context = browser.newContext();  
        Page page = context.newPage();  
        page.navigate("https://playwright.dev/java/");  
        String title = page.title();  
        System.out.println(title);  
    }  
}
```

```
    browser.close();  
    playwright.close();  
  }  
}
```

for chrome browser we need add **set channel** using **launch options**.

Chrome Browser:

Closing order (best practice)

1. browser.close() → closes browser + tabs
2. playwright.close() → shuts down engine

browser.close() closes the browser instance and all its pages, while **playwright.close()** shuts down the Playwright engine and underlying WebSocket connection. Both are required for proper resource cleanup.

Note:

By default, playwright launches browser in headless mode until we add launch options.

Playwright Debugging & CodeGen Inspector:

Codegen allows us to:

- Open a real browser, Perform actions manually and Automatically generate Playwright code
- In real life we used it to learn Playwright syntax, Identify **best selectors**, Quickly validate flows and Create POC scripts

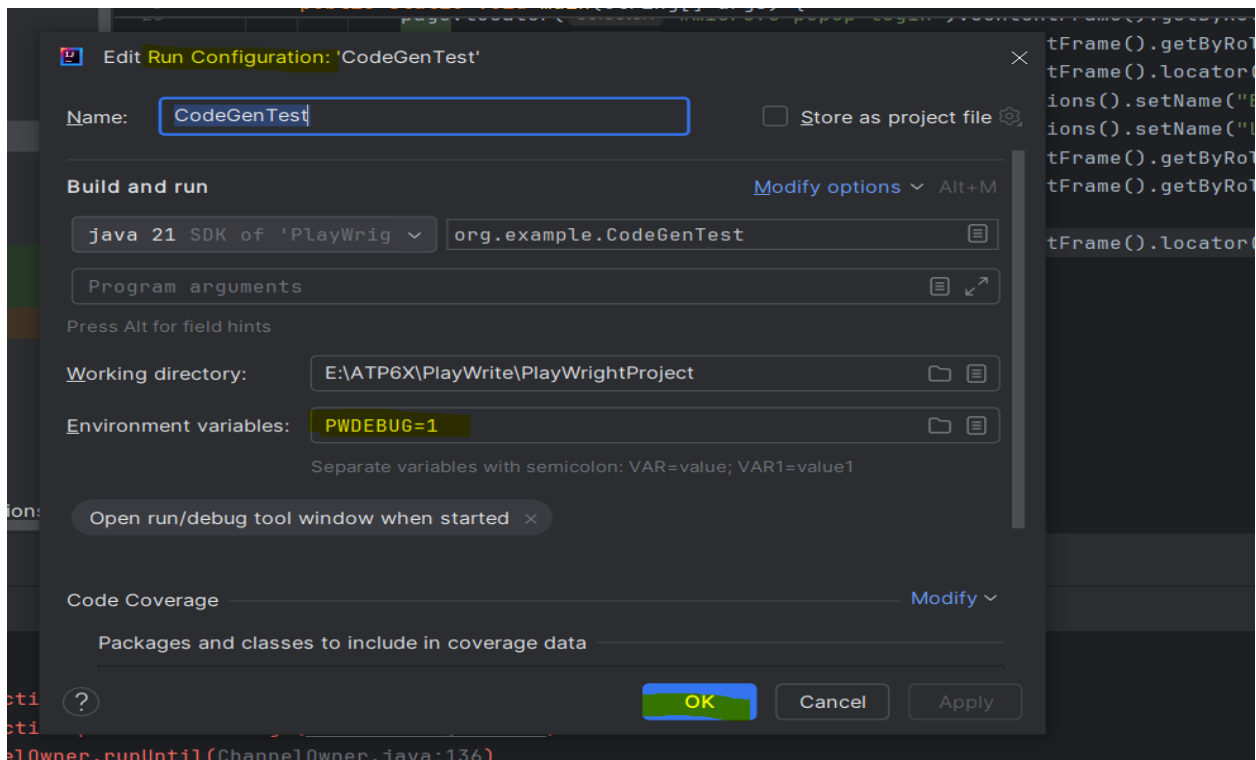
Debug Mode (PWDEBUG=1):

What happens when PWDEBUG=1 is enabled

- Browser opens in **headed mode**
- Playwright Inspector opens automatically
- Execution **pauses before every action**
- You can:
 - Step through code
 - Inspect DOM
 - Try selectors
 - Resume execution manually

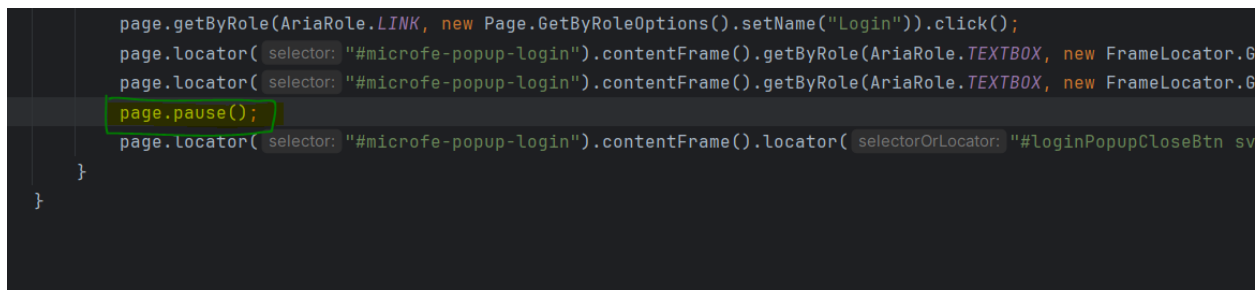
Test Engineers use it for:

- Add PWDEBUG=1 in Run Configurations
- Debug failing or flaky tests
- Understand why Playwright waits or retries



page.pause() – Manual Debug Breakpoint:

`page.pause()` we will add this in our code simply run it no need to run it in debug mode it will automatically open in debug mode once it reaches the specific debug point.



What it Does:

- Pauses test execution **at that exact line**
- Opens Playwright Inspector
- Lets us:
 - Perform actions manually
 - Add new steps
 - Copy generated code

- Continue execution

Why it's powerful

- Debug **from anywhere**
- Not tied to start of test
- Better than Java debugger for UI issues

Slow Motion:

```
launchOptions.setSlowMo(1000);
```

- Adds delay between actions
- Helps visually track execution
- Useful for demos & flaky issues

Maven Command line:

```
mvn exec:java -e -D exec.mainClass=com.microsoft.playwright.CLI -D exec.args="codegen demo.playwright.dev/todomvc"
```

Command / Option	Meaning	Explanation
mvn	Maven command-line tool	Used to build, manage dependencies, and run Java applications
exec:java	Execute Java program	Runs a Java <code>main()</code> class using Maven classpath and dependencies
-e	Error details flag	Displays full stack trace if Maven execution fails
-D	Define system property	Used to pass key-value arguments to Maven or Java runtime
exec.mainClass	Main class property	Specifies which Java class should be executed
com.microsoft.playwright.CLI	Playwright CLI class	Internal Playwright Java command-line interface
exec.args	Arguments property	Passes command-line arguments to the Playwright CLI
codegen	Code generator command	Starts Playwright Inspector and records user actions
demo.playwright.dev/todomvc	Target application URL	Opens the given URL in browser for recording
" " (quotes)	Argument grouping	Treats multiple words as a single argument
: (colon)	Plugin separator	Separates Maven plugin name and goal (<code>exec</code> → <code>java</code>)

Summary: Playwright provides powerful debugging tools such as Inspector, codegen, `page.pause()`, trace viewer, and headed execution. Codegen is mainly used for selector discovery and debugging rather than final test creation. Maven exec commands allow running Playwright's CLI directly from Java projects.

Trace Viewer:

Trace Viewer is a **debugging and analysis tool** in Playwright that records **everything that happens during a test** and allows you to **replay it later step-by-step**.

We can open traces **locally** or in your browser on **trace.playwright.dev**.

To open in Local:

```
mvn exec:java -e -D exec.mainClass=com.microsoft.playwright.CLI -D exec.args="show-trace trace.zip"
```

By default trace zip save in out project

Trace code:

```
Browser browser = browserType.launch();
BrowserContext context = browser.newContext();
```

// Start tracing before creating / navigating a page.

```
context.tracing().start(new Tracing.StartOptions()
    .setScreenshots(true)
    .setSnapshots(true)
    .setSources(true));
```

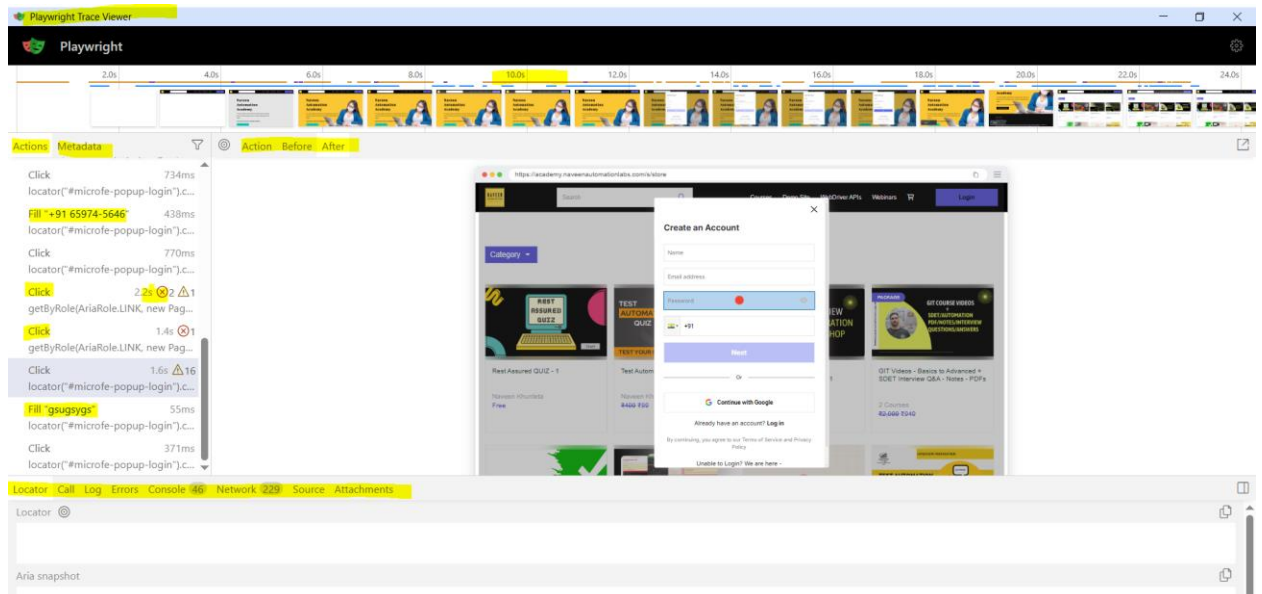
```
Page page = context.newPage();
page.navigate("https://playwright.dev");
```

// Stop tracing and export it into a zip archive.

```
context.tracing().stop(new Tracing.StopOptions()
    .setPath(Paths.get("trace.zip"));
```

Option	One-word Meaning	Explanation
<code>setScreenshots(true)</code>	Visuals	Captures screenshots at each test step to visually replay execution
<code>setSnapshots(true)</code>	DOM	Stores DOM snapshots to inspect page structure at every action

Option	One-word Meaning	Explanation
<code>setSources(true)</code>	Code	Attaches test source code to map actions with exact test lines



Trace Viewer mainly used when:

- A test **fails**
- A test is **flaky**
- A failure happens **only in CI**
- You cannot reproduce the issue locally

Trace viewer records test action, screenshot based on timeline, DOM snapshots , network activity, console log and source code.

Scenario	Why Trace Viewer helps
CI-only failures	Replay test locally
Flaky tests	See timing issues
Dynamic UI	Inspect DOM snapshots
Cross-browser issues	Compare behavior
Authentication problems	Inspect cookies & storage

Playwright Trace Viewer records a complete execution trace including actions, screenshots, DOM snapshots, and network activity. It allows replaying tests step-by-step, making it extremely effective for debugging flaky and CI-only failures.

Browser Context:

Think of **BrowserContext** = a fresh, isolated browser profile.

Each BrowserContext is like opening a **brand-new Incognito window** inside the *same browser process*.

Browser always creates in incognito mode because Playwright intentionally creates contexts with no shared data.

```
public class BrowserContextConcept {  
    public static void main(String[] args) {  
        Playwright playwright = Playwright.create();  
        Browser browser = playwright.chromium().launch(new BrowserType.LaunchOptions().setHeadless(false));  
  
        BrowserContext brcx1 = browser.newContext();  
        Page p1 = brcx1.newPage();  
        p1.navigate("https://www.orangehrm.com/orangehrm-30-day-trial");  
        p1.fill("#Form_submitForm_Name", "Naveen");  
        System.out.println(p1.title());  
  
        BrowserContext brcx2 = browser.newContext();  
        Page p2 = brcx2.newPage();  
        p2.navigate("http://automationpractice.com/");  
        p2.fill("#search_query_top", "Dress");  
        System.out.println(p2.title());  
  
        BrowserContext brcx3 = browser.newContext();  
        Page p3 = brcx3.newPage();  
        p3.navigate("http://google.com/");  
        System.out.println(p3.title());  
  
        // p1.close();  
        // brcx1.close();  
        //  
        // p2.close();  
        // brcx2.close();  
        //  
        // p3.close();  
        // brcx3.close();  
        //  
    }  
}
```

Each BrowserContext has its own:

- Cookies
- LocalStorage / SessionStorage
- Cache
- Login sessions
- Permissions

One browser Many context

- You launch **one browser** (Chromium / Firefox / WebKit)
- Inside that browser, you create **multiple BrowserContexts**

So:

- ❌ Not multiple browsers
- ✅ Multiple *isolated environments* inside one browser

Context	Acts like
Context 1	User A (not logged in)
Context 2	User B (already logged in)
Context 3	Admin user

Level	Component	What it represents	Real-life analogy	Key characteristics
1	Browser	The actual browser engine (Chromium / Firefox / WebKit)	Opening Chrome application	• Launched once • Manages resources • Cannot perform actions directly
2	BrowserContext	Isolated browser session	Incognito window	• Always clean state • No shared cookies/cache • Multiple contexts can run in same browser • Used for parallel testing
3	Page	A single tab/window	Browser tab	• Actual UI interaction happens here • Navigate, click, fill, assert • Belongs to one BrowserContext

Locators:

Locator in Playwright is a **smart, auto-waiting reference** to one or more elements on the page.

- Locator **does NOT store the element**
- Locator stores **how to find the element**
- Element is resolved **every time an action is performed**

By using Locator we can avoid:

- Stale element exceptions
- Manual waits
- Flaky tests

Playwright introduced locators to overcome below issues:

Problem in Selenium	How Locator Solves It
StaleElementException	Locator re-queries DOM
Manual waits	Auto-wait built-in
Timing issues	Retry until stable
Heavy sync code	Cleaner test code

Locators VS Web Elements:

Feature	Selenium WebElement	Playwright Locator
Stored element	Yes	No
Auto-wait	✗	✓
Retry	✗	✓
Stale element	Common	Never
Best for dynamic UI	✗	✓

How locators work internally:

1. You define a locator
2. Playwright **does NOT hit DOM immediately**
3. When you call:
 - click()
 - fill()
 - textContent()
4. Playwright:
 - Waits for element
 - Checks visibility
 - Checks stability
 - Performs action

Types of Locators in Playwright:

Category	Method	Use Case
Role-based	<code>getByRole()</code>	Best & most stable
Text-based	<code>getByText()</code>	Visible text
Label-based	<code>getByLabel()</code>	Form fields
Placeholder	<code>getByPlaceholder()</code>	Input hints
TestId	<code>getByTestId()</code>	Automation-friendly
CSS/XPath	<code>locator()</code>	Fallback option
	<code>getByTitle()</code>	
	<code>getByAlternateText()</code>	

Category	Method	Use Case
Role-based	<code>getByRole()</code>	Best & most stable

Category	Method	Use Case
Text-based	getByText()	Visible text
Label-based	getByLabel()	Form fields
Placeholder	getByPlaceholder()	Input hints
TestId	getByTestId()	Automation-friendly
CSS/XPath	locator()	Fallback option

Locators Actions:

Action	Description
click()	Click element
fill()	Enter text
type()	Type like user
clear()	Clear input
check()	Checkbox
uncheck()	Uncheck
hover()	Mouse hover
dblclick()	Double click

Frames & IFrames:

While interacting with web elements which is inside the frame tag first we need reach the tag then we will reach the web element.

```

<frameset rows="50, 290, 40" cols="400" framespacing="0" border="0">
  <frame noresize src="top.html" name="main" scrolling="no" marginwidth="12" marginheight="12">
    #document (https://www.londonfreelance.org/courses/frames/top.html)
    <html>
      <head>
      </head>
      <body bgcolor="red" background="top.gif" marginwidth="12" marginheight="12">
        <h2>
          "Title bar " == $0
        </h2>
      </body>
    </html>
  </frame>
</frameset>

```

Here my goal is to reach **Title bar h2 tag** but first I need to reach to **tag frame**.

```

Locator titleFrame=page.frameLocator("//frame[@name='main']")//reach frame
    .locator("//h2[text()='Title bar ']"); //h2 tag
System.out.println(titleFrame.textContent());

```

For I frames also we use same strategy first we navigate to **I frame** then to the required element.

```

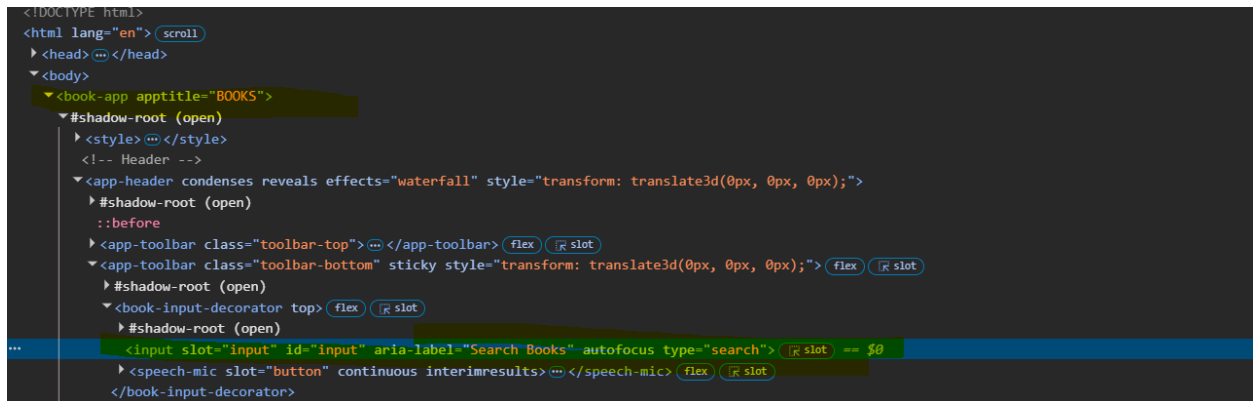
Locator lastname=page.frameLocator("//iframe[contains(@id,'frame-one')]")
    .locator("//input[@id='RESULT_TextField-3']");
lastname.fill("Suneel nare");

```

Shadow Dom:

Shadow root elements sometimes directly available inside shadow root and some times they might available iframe also.

Case 1: Page → Dom → shadow Dom → Web Element



```
page.getByLabel("Search Books").fill("latest books");
```

with `getByLabel` in playwright we directly access

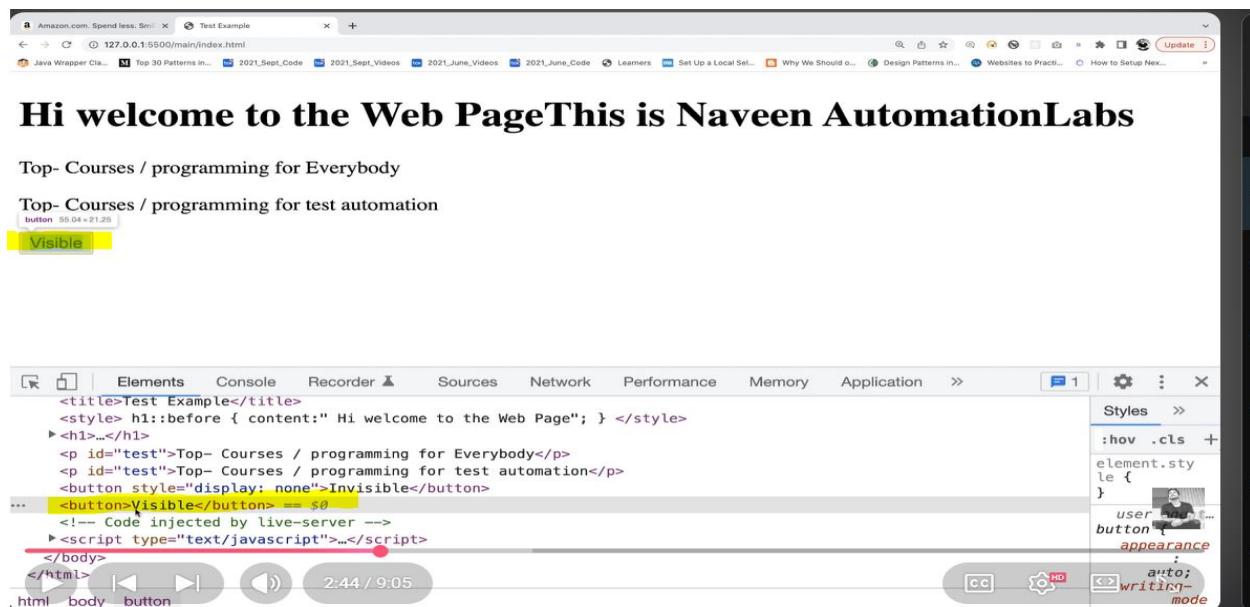
Playwright automatically handles nested Shadow DOM by `getByLabel()` and supports ARIA-based locators, making shadow DOM interaction much simpler than Selenium.

Case 2: Page → Dom → Iframe → shadow Dom → Web Element

First we need to navigate to iframe then we need to point to element.

Select Visible Elements:

Here we are interacting with the only visible element from a htm page.



```

1/
18      //button:visible
19      //button >> visible=true
20
21 //      page.navigate("http://127.0.0.1:5500/main/index.html");
22 //      //String text = page.locator("button:visible").textContent();
23 //      String text = page.locator("button >> visible=true").textContent();
24 //
25 //      System.out.println(text);
26
27      page.navigate("https://www.amazon.com/");
28      List<String> linksText = page.locator("a:visible").allInnerTexts();
29
30      for(int i=0; i<linksText.size(); i++) {
31          System.out.println(linksText.get(i));
32      }
33
34      int imagesCount = page.locator("xpath=//img >> visible=true").count();
35      System.out.println(imagesCount);
36
37      //text=Login >> visible=true
38

```

Playwright Inspector:

Directly we can interact with browser using playwright web elements so for that we need to launch our application with playwright **Codegen**. Once codegen open the url from in the **console log** we can inspect check the elements.

Give me all the links(links html tag is a) which contains text amazon:

--	--

API / Locator	What it is used for	DevTools / Inspector-style example
<code>frameLocator()</code>	Enter iframe safely (auto-waiting)	<code>playwright.frameLocator("#pact1")</code>
<code>frameLocator().locator()</code>	Locate element inside iframe	<code>playwright.frameLocator("#pact1").locator("input")</code>
<code>locator().locator()</code>	Chain locators (parent → child)	<code>playwright.locator("form").locator("input")</code>
<code>playwright.\$()</code>	First matching element (debugging only)	<code>playwright.\$("a:has-text('Amazon')")</code>
<code>playwright.\$\$()</code>	All matching elements (array, count/debug)	<code>playwright.\$\$("a:has-text('Amazon')")</code>
Playwright Inspector	Live locator validation & generation	<code>npx playwright codegen https://amazon.com</code>

DOM Element	What it contains	Best Locator to use	Why this locator
<code><p>Double-click to edit a todo</p></code>	Visible static text	<code>getByText("Double-click to edit a todo")</code>	Text is visible and stable
<code><a>Remo H. Jansen</code>	Clickable link text	<code>getByRole("link", { name: "Remo H. Jansen" })</code>	Role-based and accessible
<code><a>TodoMVC</code>	Navigation link	<code>getByRole("link", { name: "TodoMVC" })</code>	Preferred over raw text
<code><footer class="info"></code>	Structural container	<code>locator("footer.info")</code>	No role/text for web-first locators
<code><div id="concepts"></code>	Unique id	<code>locator("#concepts")</code>	ID is stable and unique
<code><div qaId="letsTest"></code>	Automation attribute	<code>getByTestId("letsTest")</code>	Dedicated test hook
<code>09</code>	Dynamic text	<code>locator(".elementor-countdown-hours")</code>	Text changes frequently
<code>Claim It Here</code>	Button text	<code>getByRole("link", { name: "Claim It Here" })</code>	User-facing action
<code></code>	Image without alt text	<code>locator("a img")</code>	<code>getByAltText()</code> not possible
<code>Products</code>	Menu item	<code>getByRole("button", { name: "Products" })</code>	Expands dropdown
<code><h4>SelectorsHub</h4></code>	Clickable card title	<code>getByRole("button", { name: "SelectorsHub" })</code>	

Sources of Accessible Name for `getByRole()`:

Source	Works with .setName()?	Notes
<label> associated with input	✓ Yes	<label for="id">Username</label> → getByRole(TEXTBOX, setName("Username")) works
aria-label attribute	✓ Yes	<input aria-label="Username"> → works
aria-labelledby attribute	✓ Yes	References another element Username
Visible inner text (buttons, links, headings)	✓ Yes	<button>Login</button> → works
Placeholder	⚠ Sometimes	<input placeholder="Username"> → may work but not reliable, depends on Playwright and browser accessibility API
name attribute	✗ No	Only used in forms, not for accessibility
class / id / type	✗ No	DOM-only attributes, ignored by getByRole()

So yes, for reliable getByRole():

- You need **either** a label, aria-label, aria-labelledby, or visible text.
- Placeholder may work but it is **not reliable**.

Xpath Union (OR condition):

CSS Selector:

```
page.navigate("https://academy.naveenautomationlabs.com");

page.locator("span:has-text('SignIn'), "
  + "span:has-text('LogIn'), "
  + "span:has-text('Login'), "
  + "span:has-text('Signin')").click();
```

X Path:

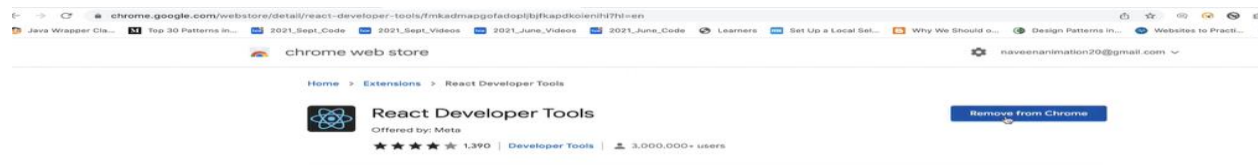
```
page.locator("//span[text()='signin'] | //span[text()='login']").click();
```

Symbol	Explanation	XPath Example
and	ALL conditions must be true on the same element	//input[@type='text' and @name='username']
or	ANY one condition can be true	//button[text()='Login' or text()='Sign in']
	Any one condition must be true in different x paths	//h2[contains(text(),'Title bar')] //p[contains(text(),'Suggest a more elegant ')]
&	Both condition must be true in different x paths	//h2[contains(text(),'Title bar')] and //p[contains(text(),'Suggest a more elegant ')]

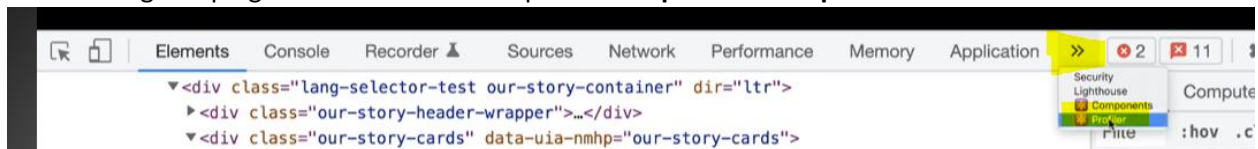
React Application Locator:

React locator is experimental and prefixed with `_`. The functionality might change in future.

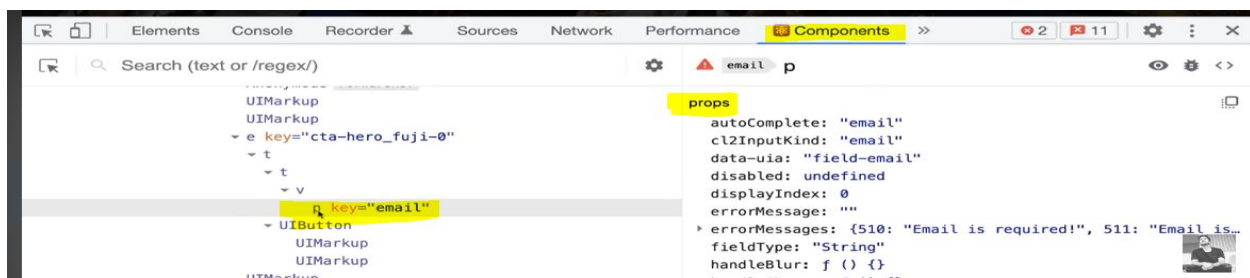
To find if a application react based or not we can use a chrome extension which is **React Developer Tools**



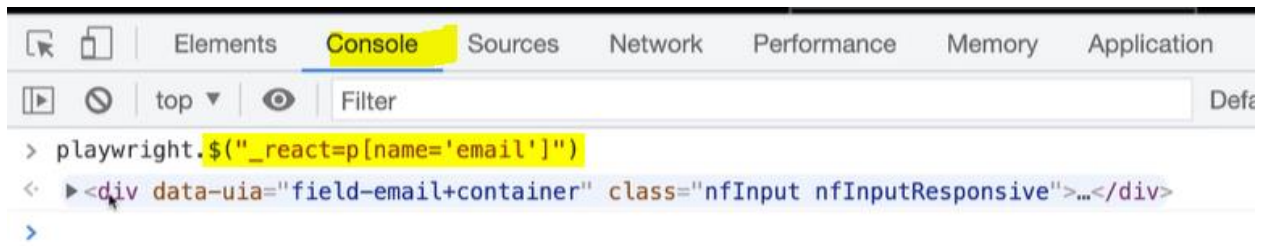
After adding this plug in it will enable two options **Components** and **profiler**.



Once we into **Components** it will show us react based selectors and **properties** with those properties, we can write react based locator paths.



React Locators:



React locator allows finding elements by their component name and property values. The syntax is very similar to **CSS attribute selectors** and supports all CSS attribute selector operators.

In **React** locator, component names are transcribed with **CamelCase**.

```
page.locator("_react=BookItem").click();
```

More examples:

- match by **component**: `_react=BookItem`
- match by component and **exact property value**, case-sensitive: `_react=BookItem[author = "Steven King"]`
- match by property value only, **case-insensitive**: `_react=[author = "steven king" i]`
- match by component and **truthy property value**: `_react=MyButton[enabled]`
- match by component and **boolean value**: `_react=MyButton[enabled = false]`
- match by property **value substring**: `_react=[author *= "King"]`
- match by component and **multiple properties**: `_react=BookItem[author *= "king" i][year = 1990]`
- match by **nested** property value: `_react=[some.nested.value = 12]`
- match by component and property value **prefix**: `_react=BookItem[author ^= "Steven"]`
- match by component and property value **suffix**: `_react=BookItem[author $= "Steven"]`
- match by component and **key**: `_react=BookItem[key = '2']`
- match by property value **regex**: `_react=[author = /Steven(\\s+King)?/i]`

XPath Locator:

```

16
17 Page page = browser.newPage();
18
19 page.navigate("https://selectorshub.com/xpath-practice-page/");
20
21 page.locator("//a[text()='Joe.Root']/parent::td/preceding-sibling::td/input[@type='checkbox']")
22     .click();
23
24
25 //
26 // //page.locator("//input[@id='twotabsearchtextbox']").fill("Macbook pro");
27 // Locator totalAmazonLinks = page.locator("//a[contains(text(),'Amazon')]");
28 // System.out.println(totalAmazonLinks.count());
29 //
30 // List<String> textList = totalAmazonLinks.allInnerTexts();
31 // for(String e : textList) {
32 //     System.out.println(e);
33 // }
34 //
35
36

```

⚠ WARNING

We recommend **prioritizing user-visible locators** like **text** or **accessible role** instead of using XPath that is tied to the implementation and easily break when the page changes.

XPath locators are equivalent to calling `Document.evaluate`.

```
page.locator("xpath=//button").click();
```

📘 NOTE

Any selector string starting with **//** or **..** are assumed to be an **xpath selector**. For example, Playwright converts `'//html/body'` to `'xpath=//html/body'`.

📘 NOTE

XPath does not pierce **shadow roots**.

XPath union

Pipe operator (`|`) can be used to specify **multiple selectors in XPath**. It will match all elements that can be selected by one of the selectors in that list.

// Waits for either confirmation dialog or load spinner.

```
page.locator("//span[contains(@class, 'spinner__loading')] | //div[@id='confirmation']").waitFor();
```

Web Table Dynamic locators:

:scope means **“current locator only”**

It tells Playwright:

- 👉 Do not search entire page
- 👉 Search only inside the current matched element

```
page.navigate("https://www.primefaces.org/primeng/table");

Locator row = page.locator("table#pr_id_2-table tr");
row.locator(":scope", new Locator.LocatorOptions().setHasText("James Butt")).locator(".p-checkbox-box")
    .click();

row.locator(":scope").allInnerTexts().forEach(e -> System.out.println(e));
```

Without :Scope :

```
page.locator("tr").locator(".select-checkbox").click();
```

Here:

- First it finds all <tr>
- Then inside them finds .select-checkbox
- But it can still behave globally depending on structure

With :Scope :

```
page = browser.newPage();
page.navigate("https://datatables.net/extensions/select/examples/initialisation/checkbox.html");

Locator row = page.locator("table#example tr");
row.locator(":scope", new Locator.LocatorOptions().setHasText("Ashton Cox")).locator(".select-checkbox").click();
row.locator(":scope").allInnerTexts().forEach(e -> System.out.println(e));
```

Here:

1. rows → all table rows
2. :scope → filter only inside each row
3. setHasText("Ashton Cox") → find row containing that text
4. Then click checkbox inside that row

It never searches entire page again.

“ :scope limits the locator search to the current matched element instead of the entire DOM.”

Step	What happens
page.locator("tr")	Gets all rows
:scope	Limits action inside each row
.locator("input")	Finds checkbox inside that specific row

Use :scope when:

- You are filtering inside an existing locator
- You want strict scoping
- You are using setHas() or setHasText() with parent locator
- You want to avoid matching outside elements

Without Scope	With Scope
May search from root	Searches inside current element
Can match outside elements	Strictly limited
Less predictable	More controlled

Playwright Authentication Using Storage State:

Normal User login:

When we login manually in browser:

1. we enter username & password
2. Server verifies credentials
3. Server sends back:
 - Cookies (session id)
 - Auth token (in localStorage or cookies)
4. Browser stores them
5. From next request → browser automatically sends cookies
6. Server says → "User already authenticated"

That's how we stay logged in.

Playwright Does the same thing:

Instead of logging in every time, Playwright:

1. Logs in once
2. Saves cookies + localStorage into a JSON file

3. Reuses that file in new browser contexts
4. Browser loads with already authenticated state
5. Application thinks user is already logged in

Flow:

Step	What Happens
1	Login normally
2	Server gives session cookies
3	Playwright saves session into JSON
4	New test loads JSON into context
5	App sees valid session
6	User is automatically logged in

Without Storage State	With Storage State
Login in every test	Login once
Slow execution	Faster tests
Repeated UI steps	Direct dashboard access
OTP issues	No OTP required
Flaky tests	Stable tests

Popup Alerts:

alert(), confirm(), prompt() -dialogs

By default, dialogs are **auto-dismissed** by Playwright, so you don't have to handle them. However, you can register a dialog handler before the action that triggers the dialog to either `Dialog.accept()` or `Dialog.dismiss()` it.

Note: it auto dismisses if a pop-up asks for conformation cancel or accept it always cancel the popups

To accept or enter prompt popups, we can add listeners. These listeners are always active they will listen to every activity when we launch a page.

Playwright provides dialog event handling using `page.onDialog()` or `page.waitForDialog()`. We can accept, dismiss, or send text for prompts. Since JS alerts are not part of DOM, we cannot use locators to interact with them.”

```

Playwright playwright = Playwright.create();
Browser browser = playwright.chromium().launch(new BrowserType.LaunchOptions().setHeadless(false));
Page page = browser.newPage();
page.onDialog(dialog -> {
    String text = dialog.message();
    System.out.println(text);
    dialog.accept("Hi this is my Alert from Naveen Automation Labs");
    //dialog.dismiss();
});

//JS Alerts, Prompt, Confirmation pop ups

page.navigate("https://the-internet.herokuapp.com/javascript_alerts");
page.click("//button[text()='Click for JS Prompt']");

String result = page.textContent("#result");
System.out.println(result);

page.close();
browser.close();
playwright.close();

```

With onDialog we can accept the alert or we can dismiss. If we don't add any dialog, the playwright's auto dismissed the pop-ups.

- **alert()** → auto-closed
- **confirm()** → automatically **dismissed** (acts like clicking Cancel)
- **prompt()** → automatically **dismissed** (acts like Cancel, no text entered)

```

//onDialog listener
page.onDialog(dialog ->{
    dialog.accept("Accepting dialogs");
});

```

Selenium	Playwright
driver.switchTo().alert()	page.onDialog()
Need to switch context	Event-based handling
Can cause NoAlertPresentException	Auto-waits for dialog

File Upload:

To upload a file we a method called **setInputFiles**.

- This will allow us to upload single file from the project file location or from any system file path.
- Multiple files from the project file location or from any system file path location,
- Allows us to create a file in run time and upload
- We can remove uploaded files

page.setInputFiles

```
setInputFiles(String selector, FilePayload files) : void - Page
setInputFiles(String selector, FilePayload[] files) : void - Page
setInputFiles(String selector, Path files) : void - Page
setInputFiles(String selector, Path[] files) : void - Page
setInputFiles(String selector, FilePayload files, SetInputFilesOptions options) : void - Page
setInputFiles(String selector, FilePayload[] files, SetInputFilesOptions options) : void - Page
setInputFiles(String selector, Path files, SetInputFilesOptions options) : void - Page
setInputFiles(String selector, Path[] files, SetInputFilesOptions options) : void - Page
```

Locator.setInputFiles() method expects first argument to point to an input element with the type "file". Multiple files can be passed in the array. If some of the file paths are relative, they are resolved relative to the current working directory. Empty array clears the selected files.

Note: if we want to upload file when inspecting the upload button, the **type=file** should be available otherwise we can upload the file. If it's not available, then we need to ask the developer to provide type=file.

```
<p>
  <strong>Upload Files:</strong>
  <input type="file" name="filesToUpload" id="filesToUpload" multiple onchange="makeFileList();" == $0
</p>
```

Single File upload from project location and system location:

```
//upload from project location
page.setInputFiles( selector: "//input[@name='filesToUpload']", Paths.get( first: "LoginAuth.json"));
//upload from system location
page.setInputFiles( selector: "//input[@name='filesToUpload']", Paths.get( first: "C:\\Users\\user\\Downloads\\12.png"));
```

Delete Uploaded file:

```
//delete uploaded file
page.setInputFiles("//input[@name='filesToUpload']", new Path[0]);
System.out.println(page.textContent("//ul[@id='fileList']"));
```

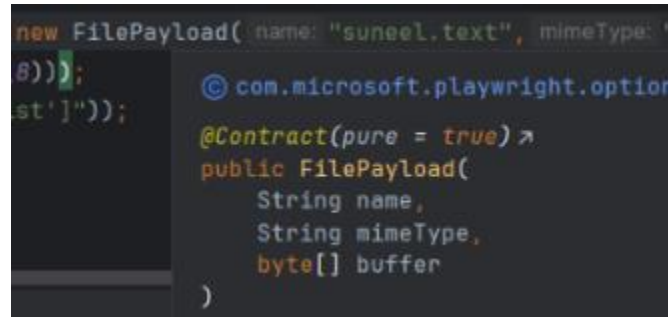
Upload Multiple files from project location and system location:

```
//upload multiple files
page.setInputFiles("//input[@name='filesToUpload']", new Path[]{
    Paths.get("C:\\Users\\user\\Downloads\\12.png"),
    Paths.get("C:\\Users\\user\\Downloads\\Capture.png"),
    Paths.get("C:\\Users\\user\\Downloads\\250.png")
});
```

Create a file on Runtime and upload:

```
//Create file on run time
page.setInputFiles("//input[@name='filesToUpload']", new
FilePayload("suneel.text", "text/plain",
    "Run time file upload".getBytes(StandardCharsets.UTF_8)));
System.out.println(page.textContent("//ul[@id='fileList']"));
```

FilePayload requires a string , mime type and buffer.



```
new FilePayload( name: "suneel.text", mimeType: "
8)));
st']"));
© com.microsoft.playwright.option
@Contract(pure = true)
public FilePayload(
    String name,
    String mimeType,
    byte[] buffer
)
```

"text/plain" -- this should follow strict MIMETYPE code otherwise it will fail

Download File:

For every attachment downloaded by the page, **Page.onDownload(handler)** event is emitted. All these attachments are downloaded into a **temporary folder**. You can obtain the download url, file name and payload stream using the **Download object** from the event.

Playwright provides a `waitForDownload()` method that listens for the download event triggered by an action. It returns a **Download object**, from which we can get the file name and save it to a desired location for validation."

Playwright solves this because:

- It waits automatically
 - It gives direct access to file
 - It works in headless mode
 - It avoids OS dependency

```
Download download=page.waitForDownload(() -> {
    page.locator("//a[text()='chromedriver_win32.zip']").click();
});
```

Download Methods:

Method	Explanation (Short)	Code Example
suggestedFilename()	Returns the file name suggested by server.	<code>String name = download.suggestedFilename();</code>
saveAs(Path path)	Saves the downloaded file to given location.	<code>download.saveAs(Paths.get("C:/files/report.pdf"));</code>
path()	Returns temporary file path (after download completes).	<code>Path filePath = download.path();</code>
url()	Returns the download URL.	<code>String url = download.url();</code>
failure()	Returns failure reason if download failed; otherwise null.	<code>String error = download.failure();</code>
cancel()	Cancels the download safely (won't fail if already finished).	<code>download.cancel();</code>
delete()	Deletes the downloaded file.	<code>download.delete();</code>
page()	Returns the page that	<code>Page p = download.page();</code>

	triggered the download.	
--	-------------------------	--

Selenium	Playwright
Hard to capture download	Built-in download event
Needs browser config	No special config
Checks OS folder	Handles internally
Flaky in parallel	Stable

Note: Playwright does NOT depend on system Downloads folder. It captures the download internally and gives you full control.

Full/Maximum Size Screen:

In Playwright, **there is no direct `maximize()` method like Selenium**, but we can achieve full screen or maximized window in different ways.

Why there is no maximize option in playwright:

Playwright works with **browser context + viewport size**, not OS-level window control like Selenium.

```
//way 1
Playwright playwright=Playwright.create();
Browser browser=playwright.chromium().launch(new
BrowserType.LaunchOptions().setHeadless(false)
.setArgs(Arrays.asList("--start-maximized")));
BrowserContext context = browser.newContext(
new Browser.NewContextOptions().setViewportSize(null));
Page page=context.newPage();
page.navigate("https://chromedriver.storage.googleapis.com/index.html?path=114.0.5735.90/");
```

Setting	Result
Default (no config)	1280×720 viewport
--start-maximized only	Still 1280×720 content

<code>setViewportSize(null)</code>	disables its default fixed viewport. Uses real window size
<code>setViewportSize(1920,1080)</code>	Fixed 1920×1080 viewport

Selenium	Playwright
<code>maximize()</code>	No direct maximize
OS-level control	Viewport-based control
Less flexible	More predictable for CI

“Playwright does not provide a direct maximize method. Instead, we launch the browser with the `--start-maximized` argument and set viewport to null, or explicitly set viewport size.”

Video recording:

Video recording in Playwright is very useful for debugging failed tests, especially in CI pipelines. Playwright can automatically record a video of each test execution and save it to a folder.

This helps when:

- Test fails in CI
- Flaky issue happens
- You want proof of execution
- Debugging UI timing issues

Note: Video recording is enabled at **BrowserContext level**, not at Page level.

```
Browser browser = playwright.chromium().launch(
    new BrowserType.LaunchOptions().setHeadless(false)
);

BrowserContext context = browser.newContext(
    new Browser.NewContextOptions()
        .setRecordVideoDir(Paths.get("videos/"))
        .setRecordVideoSize(800, 600)
);

Page page = context.newPage();
```

- Videos will save only after `context.close()`;

In frameworks we usually:

- Enable video only for failed tests
- Save videos in test-results folder
- Attach video in reports (Allure, Extent)

Selenium	Playwright
Needs third-party tools	Built-in support
Hard to configure	Simple context config
Not stable	Very stable

Video recording is enabled at the `BrowserContext` level using `setRecordVideoDir()`. The video is saved when the context is closed, and we can retrieve its path using `page.video().path()`."

Browser window Popups' Tabs:

In Playwright, each tab is represented as a `Page` object inside a `BrowserContext`. When a new tab opens, we use `context.waitForPage()` to capture it, and then interact with it directly without needing to switch like in Selenium.

- Each **tab = Page**
- Multiple tabs are created inside a **BrowserContext**

Selenium	Playwright
<code>switchTo().window()</code>	No switch needed
Uses window handles	Uses Page objects
Manual switching	Event-based handling

```
Page newPage = context.waitForPage(() -> {
    page.click("text=Open New Tab");
});
```

1. Click triggers new tab
2. Playwright waits
3. Returns new Page object

Now you can use:

```
newPage.navigate("https://example.com");
```

```
newPage.locator("text=Login").click();
```

Already Open Multiple Tabs

If multiple tabs already exist:

```
List<Page> pages = context.pages();
```

```
System.out.println(pages.size());
```

You can access:

```
Page secondTab = pages.get(1);
```

Close Parent and Continue in Child:

```
page.close();
```

```
newPage.bringToFront(); - Focus Tab
```

Method	Purpose
context.waitForPage()	Waits for new tab
context.pages()	Gets all open tabs
page.bringToFront()	Focus tab
page.close()	Close tab

child.waitForLoadState(); - When a new tab (child page) opens, the page may not be fully loaded immediately. So, we use this method to wait until the page reaches a specific loading state before continuing execution.

Screenshot:

“Playwright supports full-page screenshots, viewport-only screenshots, element-level screenshots, clipped region screenshots, and masking sensitive elements. These are configured using ScreenshotOptions.”

Normal Screenshot (Visible Area Only):

This captures only the currently visible viewport (what user sees). It captures only the currently visible viewport (what user sees).

```
page.screenshot(new Page.ScreenshotOptions()  
    .setPath(Paths.get("visible.png"))));
```

Full Page Screenshot

This captures the entire scrollable page.

```
page.screenshot(new Page.ScreenshotOptions()  
    .setPath(Paths.get("fullpage.png"))  
    .setFullPage(true));
```

What happens:

- Playwright scrolls internally
- Captures entire page
- Merges into single image

Very useful for long dashboards.

Screenshot of Specific Locator (Most Useful in Real Projects)

Capture only a selected element:

```
page.locator("#loginButton")  
    .screenshot(new Locator.ScreenshotOptions()  
        .setPath(Paths.get("element.png")));
```

- Only that element is captured
- No extra background

Very powerful for validation.

Masking Sensitive Elements

You can hide certain elements while taking screenshot:

```
page.screenshot(new Page.ScreenshotOptions()  
    .setPath(Paths.get("masked.png"))  
    .setMask(Arrays.asList(page.locator("#password"))));
```

Useful when password or sensitive data is visible.

Type	Method	When To Use
------	--------	-------------

Visible	<code>page.screenshot()</code>	Quick debug
Full Page	<code>setFullPage(true)</code>	Long pages
Element	<code>locator.screenshot()</code>	Validate specific component
Clip	<code>setClip()</code>	Custom region
Mask	<code>setMask()</code>	Hide sensitive data